Project 2: Turbojet Engine Design/Off-Design Performance Analysis

Riley J. Kyle*
Missouri University of Science and Technology, Rolla, Missouri, 65409, United States

Dr. David W. Riggins†
Missouri University of Science and Technology, Rolla, Missouri, 65409, United States

The purpose of this project is to plot the performance three space of a fixed-area missile-class engine with a simple converging nozzle. This will be done by varying the fuel flow rate from 20% to 500% the on design fuel rate, varying the Mach number from 0 to 2.5. Additionally, this will be done at sea level, 4000 m, 8000 m, and 12000 m altitude. At the design point the thrust is 306 N and the fuel flow rate is 0.0087 kg/s. Using temperature and pressure data at each trajectory point as well as a linear variation in Mach number and fuel throttle setting the design space was found numerically using a code developed in python. These results were then bound by the maximum and minimum values allowed by the engine control system. Compressor ratios higher than 18.0 or lower than 7.5, shaft RPM greater than 66000, and turbine entrance total temperatures higher than 1800K were removed, as well as some non-physical results.

Nomenclature

T = temperature $[K]$
P = pressure $[Pa]$
ρ = density $\left[kg/m^3\right]$
T_t = total temperature $[K]$
P_t = total pressure $[Pa]$
τ = $\frac{T_{t,out}}{T_{t,in}}$
π = $\frac{P_{t,out}}{P_{t,in}}$
η = isentropic efficiency
u = velocity $[m/s]$
M = mach number
A = area $\left[m^2\right]$
F = thrust $[N]$

Subscripts

d = diffuser

*Aerospace Engineering Student, MAE
†Professor of Mechanical and Aerospace Engineering, MAE

c = compressor

b = burner

t = turbine

n = nozzle

I. Introduction

The purpose of this project is to plot the performance three space of a fixed-area missile-class engine with a simple converging nozzle. This will be done by varying the fuel flow rate from 20% to 500% the on design fuel rate, varying the Mach number from 0 to 2.5. Additionally, this will be done at sea level, 4000 m, 8000 m, and 12000 m altitude. Temperature and pressure date was only given for the 8000 m case so standard atmosphere curve fits were used for the other trajectory points. The temperature and pressure information is shown in table 1. This data was used when running the solver at each altitude.

Table 1 Freestream Conditions

Altitude	T_0 (K)	P_0 (Pa)
Sea Level	288.15	101325
4000 m	262.15	61640
8000 m	236.23	35651
12000 m	216.65	19330

In this project we will be utilizing the assumption of the working fluid being a calorically perfect gas, air, with a constant C_p, γ, and R of 1004.5 J/kgK, 1.4, and 287 J/kgK respectively. It is also stated that this is a FAT jet that is choked at the turbine face and nozzle throat. This tells us that A_4, A_8, τ_t, and π_t are constant across the design space. Additionally, it was assumed that the fuel flow rate is small, and the turbine efficiency does not change; As shown in previous assignments, even without this assumption the results only change slightly.

The compressor isentropic efficiency and corrected rpm are given by the following equations:

$$\eta_c = -2.3219 + 1.7094\left(\frac{T_{t4}}{T_{t2}}\right) - 0.2897\left(\frac{T_{t4}}{T_{t2}}\right)^2 + 0.0155\left(\frac{T_{t4}}{T_{t2}}\right)^3 \tag{1}$$

$$\eta_c = 22352.47 + 3718.445\pi_c - 52.5576\pi_c^2 - 1.0899\pi_c^3 \tag{2}$$

II. Completely define the on-design engine fluid dynamics and performance

A. Methodology

The given engine information for the on design point is shown in table 2. Also, for this case we are to assume that there us full mass capture.

Table 2 On Design

Altitude	8000	m
M_0	0.85	
T_0	236.23	K
P_0	35651	Pa
M_2	0.5	
A_1	0.00318	m^2
π_c	14.43	
π_b	1.0	
π_c	0.989163	
η_c	0.90	
T_{t4}	1500	K

B. Results

Table 3 On-Design Operation

		Station						
		0	1	2	3	4	5	8
u	m/s	261.873	261.873	160.826	160.826	708.696	566.957	621.38
P	Pa	35651	35651	47679.6	758088	431148	171131	152490
T	K	236.23	236.23	257.491	604.339	1250	993.151	960.96
M		0.85	0.85	0.5	0.32637	1	0.897506	1
ρ	kg/m^3	0.525841	0.525841	0.645192	4.37076	1.20181	0.600386	0.55291
A	m^2	0.00318	0.00318	0.00422016	0.000622961	0.000514136	0.00128645	0.00127456
P_t	Pa	57177.7	57177.7	56558.1	816133	816133	288653	288653
T_t	K	270.365	270.365	270.365	617.214	1500	1153.15	1153.15

Table 4 Quantities of Interest

τ_t		0.7687676
π_t		0.3536833
F	N	306.34619
$\dot{m}_{air}$	kg/s	0.4378983
$\dot{m}_{fuel}$	kg/s	0.0087261
f		0.0199272

III. Develop the complete performance envelope for this specific turbojet engine

A. Methodology

B. Results

Figure 1-4 show the contours for the operational space for the engine at each sea level, 4000 m, 8000 m, and 12000 m. The numerical results were verified by using the on design point. At 8000 m this method gave the same solution as the method from part 1 for the design point.

IV. Conclusion

Appendix A

```
    # A comment
    # %%
import numpy as np
def eta_c_fit(Tt2,Tt4):
    x = Tt4/Tt2
    a = -2.3219
    b = 1.7094
    c = -0.2897
    d = 0.0155
    return a + b*x + c*x**2 + d*x**3

def corr_rpm_fit(pi_c):
    x = pi_c
    aa = 22352.47
    bb = 3718.445
    cc = -52.5576
    dd = -1.0899
    return aa + bb*x + cc*x**2 + dd*x**3

gamma = 1.4
Cp = 1004.5
R = 287
h = 4.45e+7

class given:
    M0 = 0.85
    T0 = 236.23 # K
    P0 = 35651 # Pa
    M2 = 0.5
    A1 = 0.00318 # m^2
    A0 = A1

    pi_c = 14.43
    pi_b = 1.0
    pi_d = 0.989163
    eta_t = 0.90

    Tt4 = 1500 # K

u = np.zeros(9)
```

```
P = np.zeros(9)
T = np.zeros(9)
M = np.zeros(9)
rho = np.zeros(9)
A = np.zeros(9)
Pt = np.zeros(9)
Tt = np.zeros(9)

M[0] = given.M0
T[0] = given.T0
P[0] = given.P0
rho[0] = P[0]/(R*T[0])
A[0] = given.A0

u[0] = M[0]*np.sqrt(gamma*R*T[0])

tau_r = 1+(gamma-1)/2 * M[0]**2
pi_r = tau_r**(gamma/(gamma-1))

Tt[0] = T[0] * tau_r
Pt[0] = P[0] * pi_r

m_dot = rho[0] * u[0] * A[0]
m_dot

pi_r

M[1] = given.M0
T[1] = given.T0
P[1] = given.P0
rho[1] = P[1]/(R*T[1])
A[1] = given.A1

u[1] = M[1]*np.sqrt(gamma*R*T[1])



Tt[1] = Tt[0]
Pt[1] = Pt[0]

M[2] = given.M2
```

```
tau_d = 1
pi_d = given.pi_d

Tt[2] = Tt[1] * tau_d
Pt[2] = Pt[1] * pi_d

T[2] = Tt[2]/(1+(gamma-1)/2 * M[2]**2)
P[2] = Pt[2]/(1+(gamma-1)/2 * M[2]**2)**(gamma/(gamma-1))

rho[2] = P[2]/(R*T[2])

u[2] = M[2]*np.sqrt(gamma*R*T[2])

A[2] = m_dot / (rho[2] * u[2])

u[3] = u[2]

pi_c = given.pi_c
eta_c = eta_c_fit(Tt[2],given.Tt4)
tau_c =  ((pi_c**((gamma-1)/gamma)-1)/eta_c)+1

Tt[3] = Tt[2] * tau_c
Pt[3] = Pt[2] * pi_c

T[3] = (Cp*Tt[3] - u[3]**2/2)/Cp
M[3] = u[3]/np.sqrt(gamma*R*T[3])
P[3] = Pt[3]/(1+(gamma-1)/2 * M[3]**2)**(gamma/(gamma-1))

rho[3] = P[3]/(R*T[3])

A[3] = m_dot / (rho[3] * u[3])

tau_c

M[4] = 1
Tt[4] = given.Tt4

pi_b = given.pi_b

Pt[4] = Pt[3] * pi_b
```

```
T[4] = Tt[4]/(1+(gamma-1)/2 * M[4]**2)
P[4] = Pt[4]/(1+(gamma-1)/2 * M[4]**2)**(gamma/(gamma-1))

rho[4] = P[4]/(R*T[4])

u[4] = M[4]*np.sqrt(gamma*R*T[4])

A[4] = m_dot / (rho[4] * u[4])

tau_lambda = Tt[4] / T[0]

m_dot_f = m_dot * Cp * (Tt[4]-Tt[3]) / h

u[5] = 0.8 * u[4]

eta_t = given.eta_t

tau_t = 1-(tau_c - 1)*(tau_r/tau_lambda)
pi_t = (1-1/eta_t*(1-tau_t))**(gamma/(gamma-1))

Tt[5] = Tt[4] * tau_t
Pt[5] = Pt[4] * pi_t

T[5] = (Cp*Tt[5] - u[5]**2/2)/Cp
M[5] = u[5]/np.sqrt(gamma*R*T[5])
P[5] = Pt[5]/(1+(gamma-1)/2 * M[5]**2)**(gamma/(gamma-1))

rho[5] = P[5]/(R*T[5])

A[5] = m_dot / (rho[5] * u[5])

M[8] = 1

tau_n = 1
pi_n = 1

Tt[8] = Tt[5] * tau_n
Pt[8] = Pt[5] * pi_n

T[8] = Tt[8]/(1+(gamma-1)/2 * M[8]**2)
P[8] = Pt[8]/(1+(gamma-1)/2 * M[8]**2)**(gamma/(gamma-1))
```

```
rho[8] = P[8]/(R*T[8])

u[8] = M[8]*np.sqrt(gamma*R*T[8])

A[8] = m_dot / (rho[8] * u[8])

F = (m_dot) * (u[8] - u[0]) + (P[8] - P[0]) * A[8]
f = m_dot_f/m_dot

table = np.array([
    u,
    P,
    T,
    M,
    rho,
    A,
    Pt,
    Tt
])
```

Appendix B

```
import numpy as np
import scipy
import matplotlib.pyplot as plt

# %%
pi_t = 0.35368326100748115
tau_t = 0.7687676014665914
eta_t = 0.90
pi_d =  0.989163
pi_b = 1

m_dot_fR = 0.008726071272118822

A1 = 0.00318
A4 = 0.0005141364620689669
A8 = 0.0012745633749949147
M4 = 1
M8 = 1

# %%
tau_d = 1

# %%
pi_c_max = 18
pi_c_min = 7.5

rpm_max = 66000

Tt4_max = 1800

# %%
gamma = 1.4
Gamma = (2/(gamma+1))**((gamma+1)/(2*(gamma-1)))*np.sqrt(gamma)
Cp = 1004.5
R = 287
h = 4.45e+7

Alt_list = np.array([0,4000,8000,12000])
T0_list = np.array([288.15 ,262.15 ,236.23 ,216.65])
P0_list = np.array([101325 , 61640.235,35651 ,19330.405])
```

```
rho0_list = P0_list/(R*T0_list)
a0_list = np.sqrt(gamma*R*T0_list)

# %%
n = 200
Alt_size = Alt_list.size

M0 = np.linspace(0,2.5,n)
m_dot_f = np.linspace(.2,5,n) * m_dot_fR

M0, m_dot_f = np.meshgrid(M0,m_dot_f)

M0 = np.matrix.flatten(M0)
m_dot_f = np.matrix.flatten(m_dot_f)

#M0 = np.array([0.85])
#m_dot_f =  np.array([m_dot_fR])

j = 3

Alt = Alt_list[j]
T0 = T0_list[j]
P0 = P0_list[j]
rho0 = P0/(R*T0)
a0 = np.sqrt(gamma*R*T0)



Tt4_plot = np.zeros(n**2)
rpm_plot = np.zeros(n**2)
pi_c_plot = np.zeros(n**2)
Thrust_plot = np.zeros(n**2)
spill_plot = np.zeros(n**2)

def eta_c_fit(Tt4):
        x = Tt4/Tt2
        a = -2.3219
        b = 1.7094
        c = -0.2897
        d = 0.0155
        return a + b*x + c*x**2 + d*x**3
```

```
def corr_rpm_fit(pi_c):
    x = pi_c
    aa = 22352.47
    bb = 3718.445
    cc = -52.5576
    dd = -1.0899
    return aa + bb*x + cc*x**2 + dd*x**3

def _tau_c(Tt4):
    return 1+(Tt4/Tt0)*(1-tau_t)

def _pi_c(Tt4):
    return  (1+eta_c_fit(Tt4)*(_tau_c(Tt4)-1))**(gamma/(gamma-1))

def _m_dot(Tt4):
    return Gamma / np.sqrt(R) * A4 * (P0*pi_r*pi_d*_pi_c(Tt4)*pi_b)*1/(np.sqrt(Tt4))

def _mdotf(Tt4):
    return _m_dot(Tt4)*Cp*(Tt4 - _tau_c(Tt4) * Tt2)/h-m_dot_f[i]

for i in range(M0.size):
    u0 = M0[i] * np.sqrt(gamma*R*T0)

    m_dot_no_spill = rho0 * u0 * A1

    tau_r = 1+(gamma-1)/2 * M0[i]**2
    pi_r = tau_r**(gamma/(gamma-1))

    Tt0 = T0 * tau_r
    Pt0 = P0 * pi_r

    Tt2 = Tt0
    Pt2 = Pt0 * pi_d

    Tt4_guess = 800
    Tt4 = scipy.optimize.fsolve(_mdotf,Tt4_guess,maxfev=90000)[0]

    tau_c = _tau_c(Tt4)
    pi_c = _pi_c(Tt4)

    m_dot_air = _m_dot(Tt4)
```

```
    corr_rpm = corr_rpm_fit(pi_c)# np.sqrt(Tt0/T0)

    eta_c = eta_c_fit(Tt4)



    Tt8 = Tt4 * tau_t
    Pt8 = Pt2 * pi_c * pi_b * pi_t

    T8 = Tt8 / (1 + (gamma-1)/2 * M8**2)
    P8 = Pt8 / (1 + (gamma-1)/2 * M8**2)**(gamma/(gamma-1))

    u8 = M8 * np.sqrt(gamma*R*T8)

    Tt4_plot[i] = Tt4
    rpm_plot[i] = corr_rpm
    pi_c_plot[i] = pi_c
    spill_plot[i] = (m_dot_air-m_dot_no_spill)
    Thrust_plot[i] = (m_dot_air) * (u8 - u0) + (P8 - P0) * A8

    if Tt4 > Tt4_max or Tt4 <= Tt0 or Tt4 == Tt4_guess or pi_c > pi_c_max or pi_c < pi_c_min or
                                                    corr_rpm > rpm_max:
        Tt4_plot[i] = np.nan
        pi_c_plot[i] = np.nan
        rpm_plot[i] = np.nan
        Thrust_plot[i] = np.nan
        spill_plot[i] = np.nan
        continue

Tt4_plot = np.reshape(Tt4_plot,(n,n))
rpm_plot = np.reshape(rpm_plot,(n,n))
pi_c_plot = np.reshape(pi_c_plot,(n,n))
spill_plot = np.reshape(spill_plot,(n,n))
Thrust_plot = np.reshape(Thrust_plot,(n,n))
M0 = np.reshape(M0,(n,n))
m_dot_f = np.reshape(m_dot_f,(n,n))

fig = plt.figure(1)
ax = plt.axes()
cp = ax.contourf(M0,m_dot_f,Tt4_plot)
cbar = fig.colorbar(cp)
cbar.ax.set_ylabel(r'Tt4 (K)')
```

```
if j == 2:
    ax.scatter(0.85,m_dot_fR,c='k',marker='x')
ax.set_xlabel('Mach')
ax.set_ylabel(r'$\dot{m}_f \ (kg/s)$')
plt.tight_layout()
plt.savefig(r'{}_m_Tt4.png'.format(Alt_list[j]), dpi=1200)
plt.show()

# %%
fig = plt.figure(2)
ax = plt.axes()
cp = ax.contourf(M0,m_dot_f,rpm_plot,cmap='viridis')

if j == 2:
    ax.scatter(0.85,m_dot_fR,c='k',marker='x')
ax.set_xlabel('Mach')
ax.set_ylabel(r'$\dot{m}_f \ (kg/s)$')
plt.tight_layout()
cbar = fig.colorbar(cp)
cbar.ax.set_ylabel('RPM')
plt.savefig(r'{}_m_rpm.png'.format(Alt_list[j]), dpi=1200)
plt.show()

# %%
fig = plt.figure(3)
ax = plt.axes()
cp = ax.contourf(M0,m_dot_f,pi_c_plot)

if j == 2:
    ax.scatter(0.85,m_dot_fR,c='k',marker='x')
ax.set_xlabel('Mach')
ax.set_ylabel(r'$\dot{m}_f \ (kg/s)$')
plt.tight_layout()
cbar = fig.colorbar(cp)
cbar.ax.set_ylabel(r'$\pi_c$')
plt.savefig(r'{}_m_pic.png'.format(Alt_list[j]), dpi=1200)
plt.show()

# %%
fig = plt.figure(4)
```

```
ax = plt.axes()
cp = ax.contourf(M0,m_dot_f,spill_plot)

if j == 2:
    ax.scatter(0.85,m_dot_fR,c='k',marker='x')
ax.set_xlabel('Mach')
ax.set_ylabel(r'$\dot{m}_f \ (kg/s)$')
plt.tight_layout()
cbar = fig.colorbar(cp)
cbar.ax.set_ylabel(r'spill (kg/s)')
plt.savefig(r'{}_m_spill.png'.format(Alt_list[j]), dpi=1200)
plt.show()

# %%
fig = plt.figure(5)
ax = plt.axes()
cp = ax.contourf(M0,m_dot_f,Thrust_plot)

if j == 2:
    ax.scatter(0.85,m_dot_fR,c='k',marker='x')
ax.set_xlabel('Mach')
ax.set_ylabel(r'$\dot{m}_f \ (kg/s)$')
plt.tight_layout()
cbar = fig.colorbar(cp)
cbar.ax.set_ylabel(r'F (N)')
plt.savefig(r'{}_m_thrust.png'.format(Alt_list[j]), dpi=1200)
plt.show()
```